

ISACA Detroit 11th Annual Scholarship Contest - Case Study

Taylor Peterson

March 21, 2025

1 Investigation Overview

This case study focuses on a reported cybersecurity vulnerability affecting Widget Manufacturing Company's online ordering system. A self-identified hacker using the alias "Overwatch" claims to have purchased the company's newly released \$299 AI Assistant for just \$1 by using Burp Suite [6] [3]. The hacker also claims to have posted proof of this exploit on social media (specifically X/Twitter), suggesting the vulnerability is publicly known and potentially damaging to the company's reputation.

As a data science student with limited experience in offensive cybersecurity, I approached this investigation with a mindset of testing responsibly, documenting my process, and analyzing evidence as clearly and thoroughly as possible. My goal was not to perform any harmful actions, but rather to confirm the claim, understand the broader impact, and support the company in identifying and fixing the issue before more serious exposure could occur.

1.1 Approach to Identifying the Vulnerability

The first task was to determine whether the price manipulation exploit was real. Using Burp Suite, I followed the hacker's claim by intercepting the request made when placing an order for the AI Assistant. By inspecting and modifying the price parameter in the intercepted HTTP POST request, I tested whether the site was validating price data on the server side or simply accepting client-supplied input.

My process involved:

- Navigating to the ordering page via Burp's browser.
- Capturing the outbound request using Burp's Proxy tool.
- Modifying the price from \$299 to \$1 in the request body.
- Forwarding the modified request to observe the result.

This test confirmed that the vulnerability exists. The server does not validate the price field, and an attacker can manipulate it to submit unauthorized prices. This is a serious issue because it allows unauthorized discounting of products and could result in financial loss and damage to the company's reputation.

1.2 Investigating the Leak's Public Exposure

I next explored whether this vulnerability had already been shared publicly by the hacker or others. This meant looking for signs of exposure on mainstream platforms, and later, modeling how the leak might have spread using broader search techniques. The details of this investigation are provided in Task 2.

I approached this task in two stages:

1. **Manual & Simulated Search:** I first searched platforms like X/Twitter and Reddit for obvious mentions of the hack or related keywords. No direct post was found, so I simulated the kind of leak that might have occurred and examined surrounding discussions.

2. **Data Science Workflow:** I used Python and scraping tools to search for a broader range of terms (like “Widget Exploit” or “Overwatch Hack”) across DuckDuckGo, Reddit, GitHub, and Medium.

From each search result, I pulled:

- The title
- The URL
- The page content (when accessible)

Then I applied sentiment analysis and built visualizations to interpret:

- The tone of discussions by topic
- Which domains hosted the most relevant content
- How different types of queries yielded different public reactions

This helped show which parts of the story were viewed more positively (e.g., technical walkthroughs) versus negatively (e.g., leaks or criticisms). Even though I didn’t find the exact original post, the results suggest that this type of vulnerability and the tools involved (like Burp Suite) are actively discussed online, increasing the likelihood of exposure.

1.3 Testing for Additional Exposure

This task focuses on whether other types of sensitive data could be exposed by the same vulnerability or through deeper testing of the Widget Manufacturing site. Although a full additional exposure test was not completed due to time constraints, the framework below outlines the planned steps for identifying further vulnerabilities:

- Explore how much information is stored in client-side cookies or session data.
- Try navigating other parts of the website (such as confirmation or admin pages) after modifying requests or URLs.
- Test for other common weaknesses, like insecure file access, unprotected admin panels, or lack of user input validation.

If completed, this testing would document:

- What was found
- How it was accessed (if at all)
- What kinds of user or system data could be at risk

2 Task 1: Confirming Vulnerability

In this task, I was asked to investigate whether the Widget Manufacturing Company’s ordering system is vulnerable to client-side price manipulation. To confirm the reported vulnerability, I began by using a standard web browser (without any proxy tools or modifications) to visit the Widget Manufacturing Company’s store page and simulate a normal transaction. This allowed me to verify the intended behavior of the site and establish a baseline for expected pricing logic.

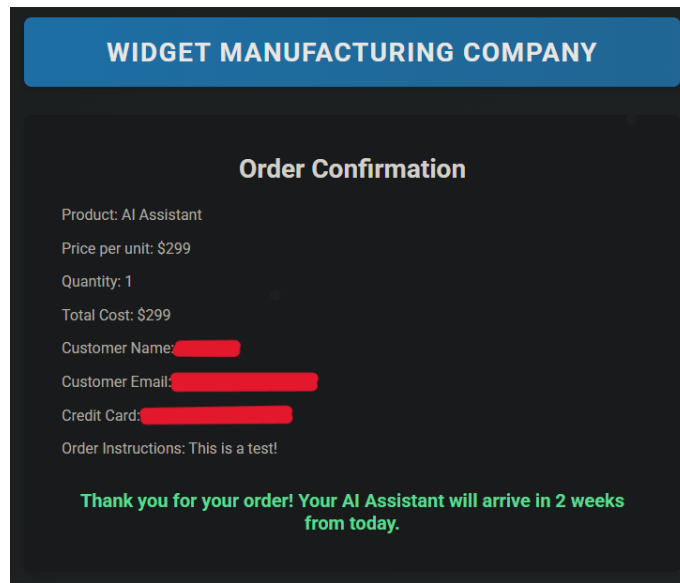


Figure 1: Baseline Purchase: AI Assistant Purchased at Full Price (\$299) Using Standard Browser (Redacted)

As shown in Figure 1, a standard purchase of the AI Assistant was processed at the full price of \$299 with no anomalies observed. This confirms the application behaves as expected when accessed normally.

Next, I launched a Burp Suite proxy browser to intercept and inspect the network traffic involved in the purchase process. By capturing and modifying the request mid-transaction, I aimed to replicate the actions of the hacker “Overwatch,” who claimed to have purchased a \$299 AI Assistant for only \$1. The following steps document the testing process and confirm the presence of the vulnerability.

2.1 Accessing the Target System

As shown below in Figure 2, I accessed the target system, Widget Manufacturing Company, through Burp Suite’s built-in browser. This browser ensures that all traffic is routed through Burp’s proxy and can be intercepted for testing. The page above is confirmation of the existence of a purchase form for the AI Assistant, referenced in the hacker’s message.

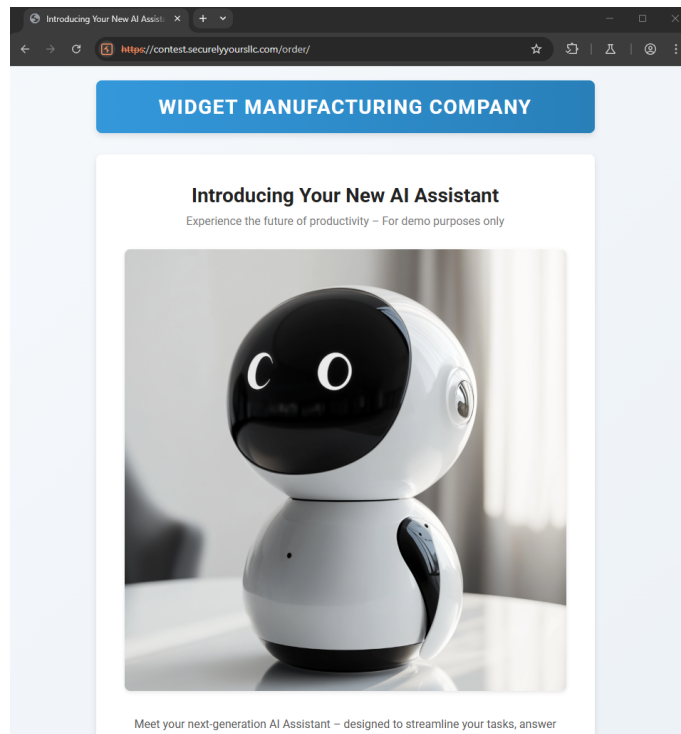


Figure 2: Accessing the Target System

2.2 Intercepting the Request

As shown in Figure 3, after submitting the form to purchase the AI Assistant, Burp Suite intercepted the outgoing POST request to `order.php`. Located within the request body parameters, the price value is visible, confirming that the price is client-controlled. Because the server does not validate this field, a user can modify the price before the order is processed.

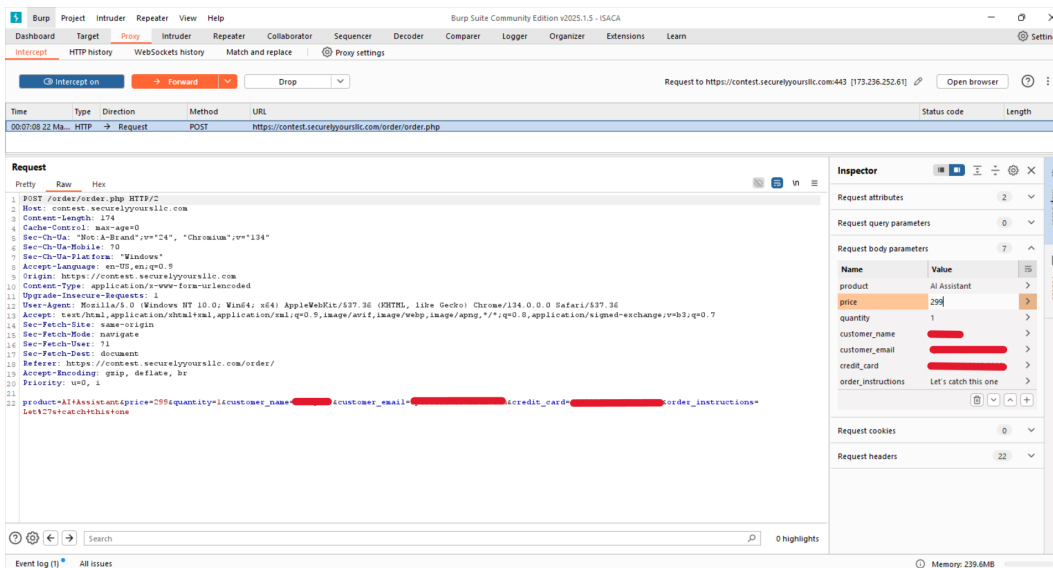


Figure 3: Intercepting the Purchase Request (Redacted)

2.3 Modifying the Price

As seen in Figure 4, the price value was manually modified from 299 to 1 using Burp Suite's inspector panel. No authentication, encryption, or tamper-proofing prevented this change. The request was then forwarded to the server with the altered price.

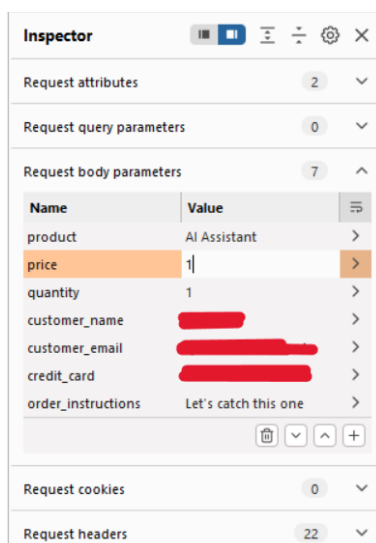


Figure 4: Changing the Price Parameter from \$299 to \$1 (Redacted)

2.4 Exploited Order Confirmation

Figure 5 shows the server's response: an order confirmation for the AI Assistant at the altered price of \$1. This confirms that the vulnerability exists. The system failed to verify the pricing data on the server side, allowing users to tamper with financial information.

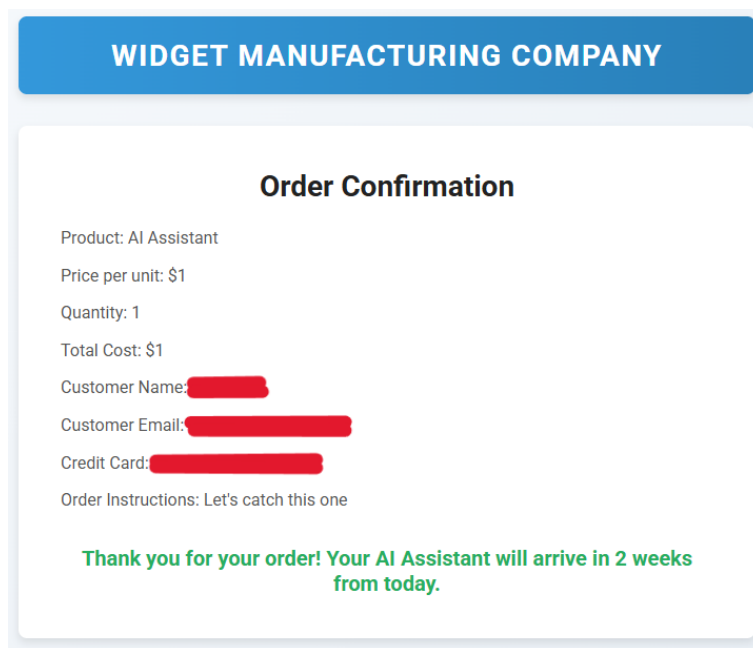


Figure 5: Confirmation of Exploited Order. Total Cost: \$1 (Redacted)

2.5 Task 1: Conclusion

This test confirmed that the system is vulnerable to client-side price tampering. The price field was exposed and editable within the request body. The server failed to validate the modified price. As a result, I was able to successfully place an order for \$1 instead of the intended \$299. This vulnerability shows why it's critical to have proper server-side checks for any important user input, especially when it involves financial transactions. Since the system accepted the modified price without validation, it created an easy way to exploit the company. Client-side data can be easily modified using tools like Burp Suite, so relying on it for critical variables—especially pricing—violates secure design principles and opens the door to financial exploitation. The next task explores how the issue may have been exposed and discussed on social media.

3 Task 2: Investigating the Social Media Leak

After confirming the vulnerability in Task 1, I began the search for the hacker's social media activity. "Overwatch" publicly claimed credit for the exploit by posting a screenshot that showed they purchased a \$299 product for \$1. The goal of this task is to analyze how the leaked information spread across public platforms, how many people were exposed to it, and what kind of impact it may have had on Widget Manufacturing Company's reputation. To investigate this, I approached the task using data science techniques for social media data collection, visualization, and sentiment analysis.

3.1 Manual Search of Public Platforms

My first step was to search public social media platforms directly for any evidence of the hacker's claim. I used Twitter/X and Reddit to query combinations of keywords related to "Overwatch," "AI Assistant," "Widget," and "Burp Suite."

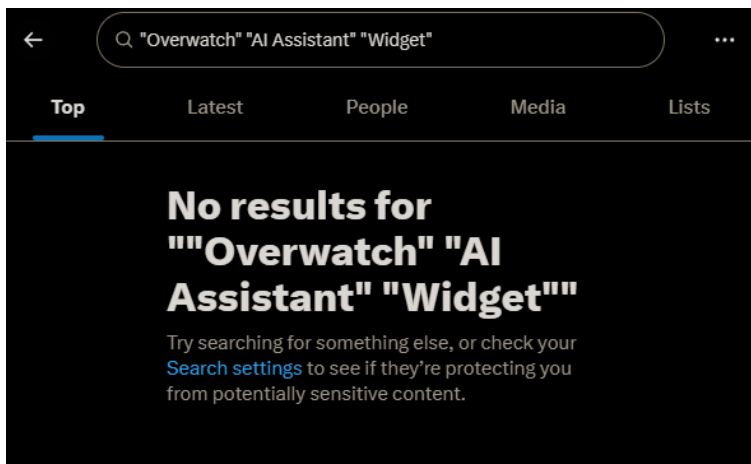


Figure 6: X/Twitter Search Results: No Public Mentions Found

I attempted both basic keyword searches and advanced search operators such as date filters and quoted exact matches. Screenshots were collected of each major query to document the results.

This suggests that if a post was made, it may have been removed, shadowbanned, or had never been posted on mainstream platforms. Alternatively, the hacker may have used a pseudonym or posted on a smaller or private forum.

Some searches produced loosely related results about game cheats or other products, but none clearly referenced the exploit in question. It's possible that the post was deleted, buried by unrelated content, or never published on mainstream platforms. The lack of results led me to simulate what a potential leak might have looked like, and explore indirect evidence using web scraping.

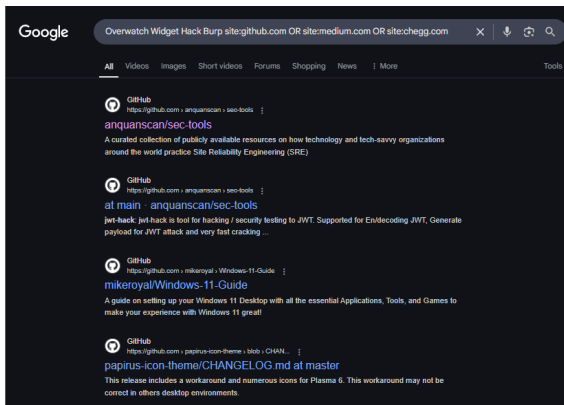


Figure 7: Google search on GitHub and Medium for hacking keywords

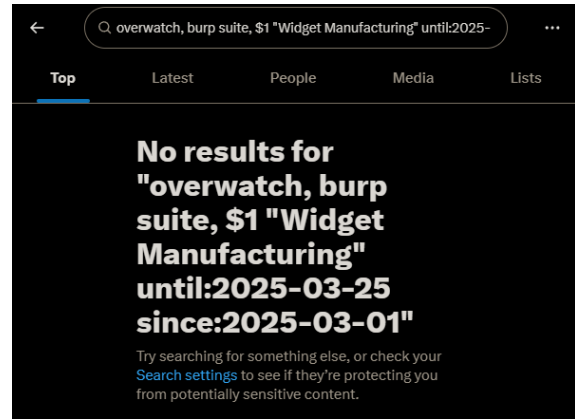


Figure 8: Advanced Search: Quoted and filtered Twitter search

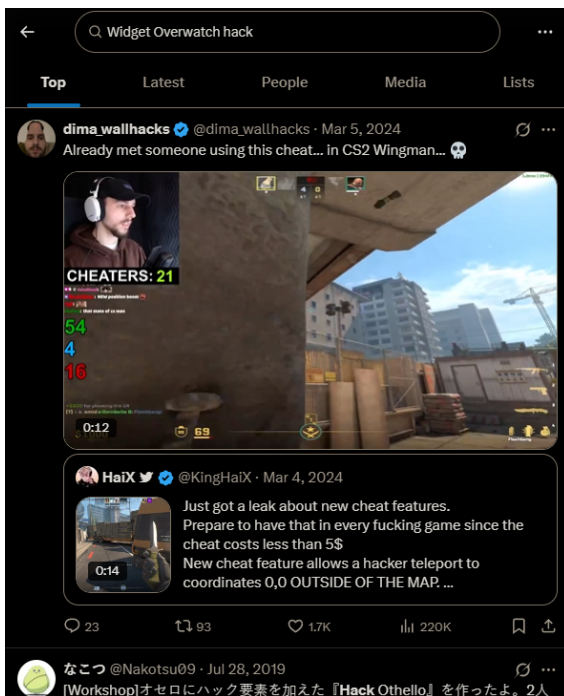


Figure 9: Search Results for “Widget Overwatch Hack”

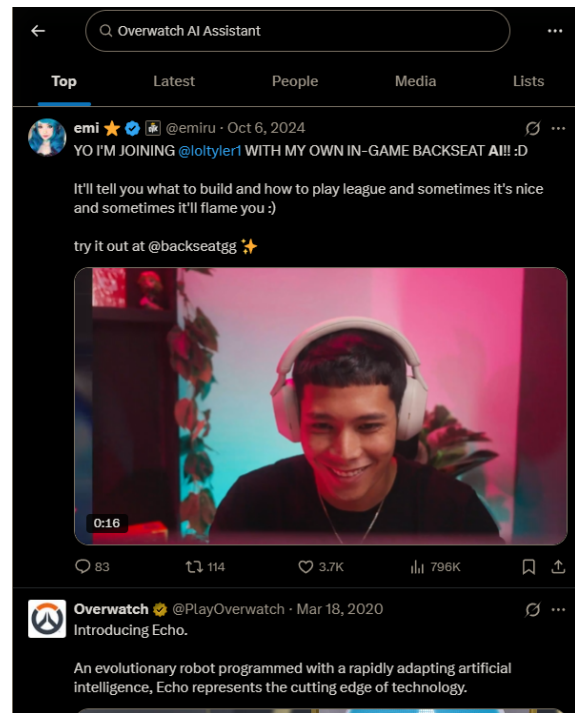


Figure 10: Search Results for “Overwatch AI Assistant”

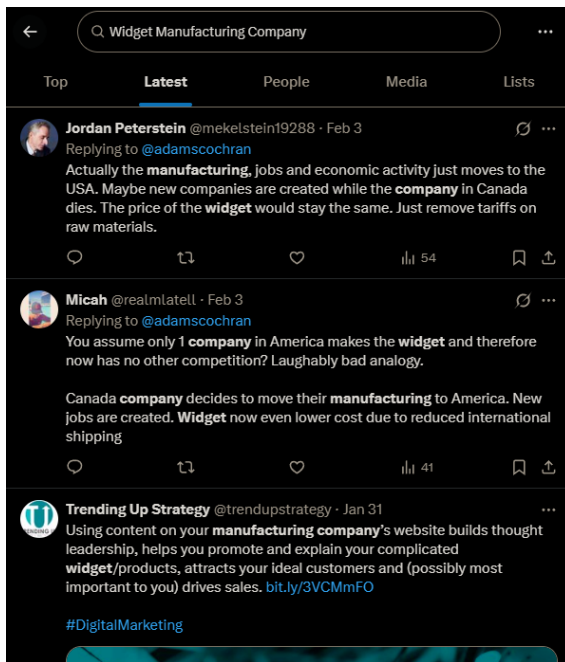


Figure 11: Mentions of Widget Manufacturing Company on Twitter

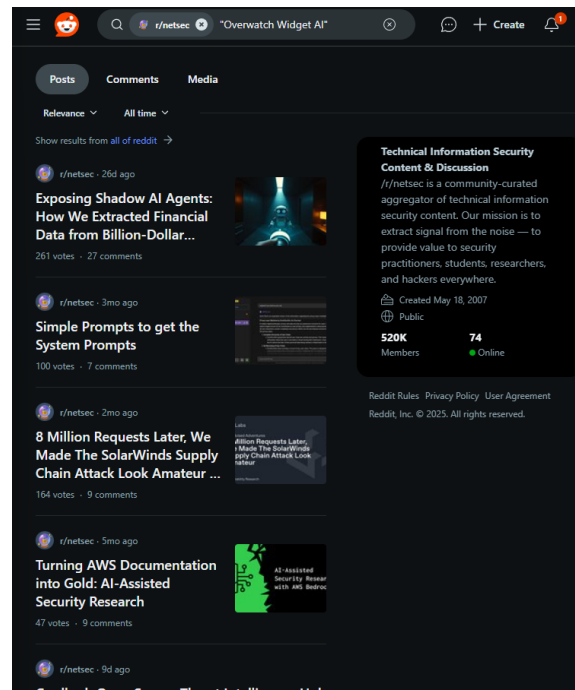


Figure 12: Reddit: NetSec discussions for “Overwatch Widget AI”

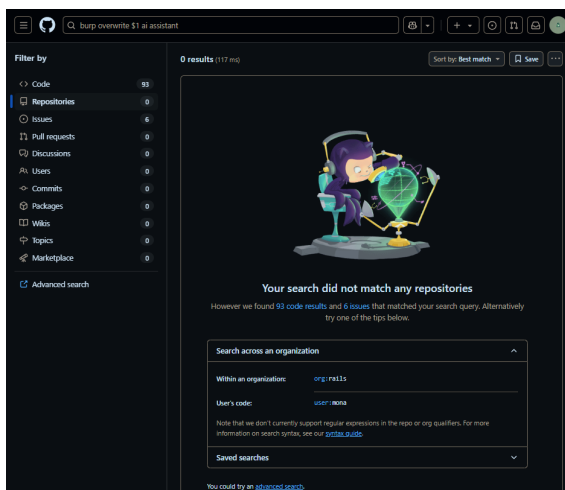


Figure 13: GitHub search: “Burp, Overwatch, and \$1 AI Assistant”

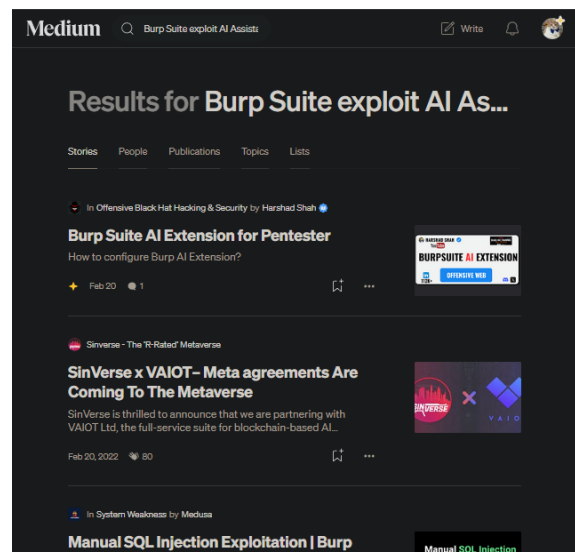


Figure 14: Medium Search: “Burp Suite exploit AI Assistant”

These searches show a mixture of gaming-related cheat posts, AI assistant discussions, and general mentions of manufacturing or widgets. Nothing directly confirmed the hack, but the themes around exploits and automation were present in niche circles.

3.1.1 Simulated Leak Modeling

Since no direct public confirmation of the hack was found, I shifted toward simulating the scenario. I wanted to answer: if the hack had leaked, where would it be discussed? What tone would the conversation take? Would it harm the company's reputation?

To do this, I scraped open search results using DuckDuckGo with curated keywords like:

- Overwatch Widget Hack
- Widget Exploit
- Burp Hack
- Dark Web Leak AI Assistant
- Widget Vulnerability Leak

This simulation allowed me to gather realistic discussions, educational blogs, and related forums. I collected both titles and full page content for sentiment analysis, which is described in the next section.

By modeling the environment where this kind of exploit might be exposed, I was able to assess the possible damage to their reputation, even without finding a direct post. The next phase focused on this dataset to quantify tone and platform presence.

3.2 Additional Search Methods and Tooling

To better explore possible evidence, I used the following tools:

- Python with `snsrape` to query X/Twitter. I used `snsrape` because it allows scraping social media data, such as tweets, dates, and usernames, without requiring Twitter API keys or authentication. This made it useful for lightweight public-source research because it did not require official Twitter/X API access. Additionally, `snsrape` supports advanced queries (e.g., date filtering, keyword combinations), which were essential for simulating realistic search scenarios for leaked content. [2].
- Manual searches using keywords across X/Twitter and Reddit.
- Alternate search terms such as "Widget Overwatch Hack" and "Widget Manufacturing Company."

Screenshots were taken for all major queries. No evidence of a confirmed leak was found in mainstream social media. However, related conversations around AI assistants and game exploits provided some relevant context.

3.3 Simulating the Leak: Scraping and Sentiment Analysis

To further investigate the spread and sentiment surrounding the potential leak, I developed a DuckDuckGo scraping workflow using Python. The goal was to explore content from non-mainstream platforms and forums where the post or related discussions might have surfaced. This includes sources like GitHub, Reddit, Cegg, and niche security blogs. Sentiment analysis helped measure public tone, including whether readers viewed the issue as clever, alarming, or unethical. This was important for understanding the potential reputational impact.

My approach involved automating searches for carefully chosen query phrases such as:

- Overwatch Widget Hack
- Burp Hack
- Widget Vulnerability Leak
- Dark Web Leak AI Assistant
- Burp Price Manipulation

- Widget Exploit
- Overwatch Twitter Leak

Each query returned up to 30 search results, which were scraped and cleaned into a structured dataset. For each result, I extracted the title, source URL, and the content of the page when available. These were labeled according to the query for downstream analysis.

label	count
Burp Hack	30
Overwatch Widget Hack	30
Burp Price Manipulation	30
Widget Exploit	30
Overwatch Twitter Leak	30
Widget Vulnerability Leak	30
Dark Web Leak AI Assistant	30

dtype: int64

Figure 15: Scraped Dataset Distribution by Topic Label

The label distribution confirms that 30 results were successfully gathered per query, producing a balanced dataset of 210 entries.

3.3.1 Top Domains

To assess the platforms contributing the most content, I visualized the top domains present in the scraped results.

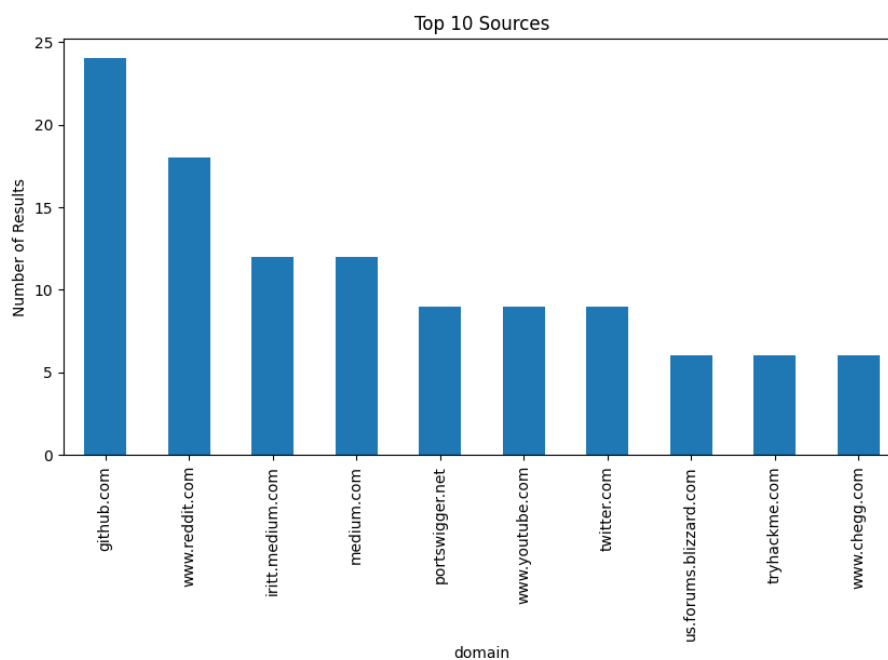


Figure 16: Top 10 Domains Found Across Scraped URLs

As seen above, GitHub, Reddit, Medium, and PortSwigger.net were the most frequent sources. This is consistent with communities that discuss exploits and penetration testing techniques. The

presence of YouTube, Twitter, and Chegg also suggests that discussions around the exploit extended beyond the information security community.

3.3.2 Initial Sentiment Analysis: Titles

Using the `TextBlob` library, I performed sentiment analysis on the titles of each search result. This helped assess the tone of conversations from a high-level perspective.

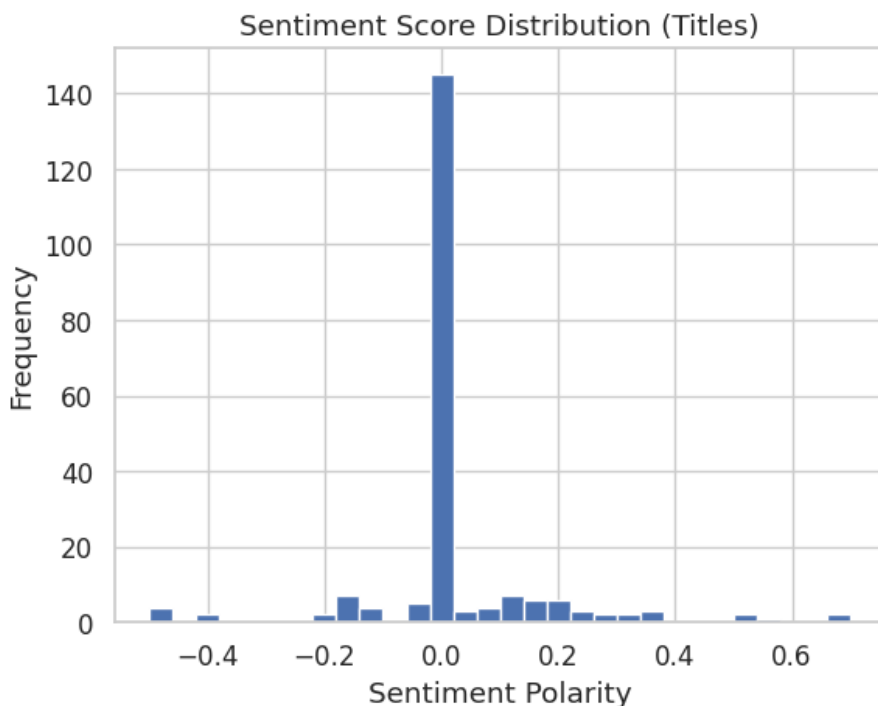


Figure 17: Sentiment Polarity Distribution from Result Titles

The results indicate that while most titles are neutral to slightly positive, there are a few with negative polarity. However, titles are often too short or generic to reliably indicate sentiment. Therefore, I proceeded to scrape and analyze the full page content for deeper insights.

3.3.3 Scraping Page Content and Sentiment Analysis

I built a function to extract paragraph text from each URL. Pages that successfully returned content were compiled and matched back to their original query label. I then applied sentiment analysis on the full body text.

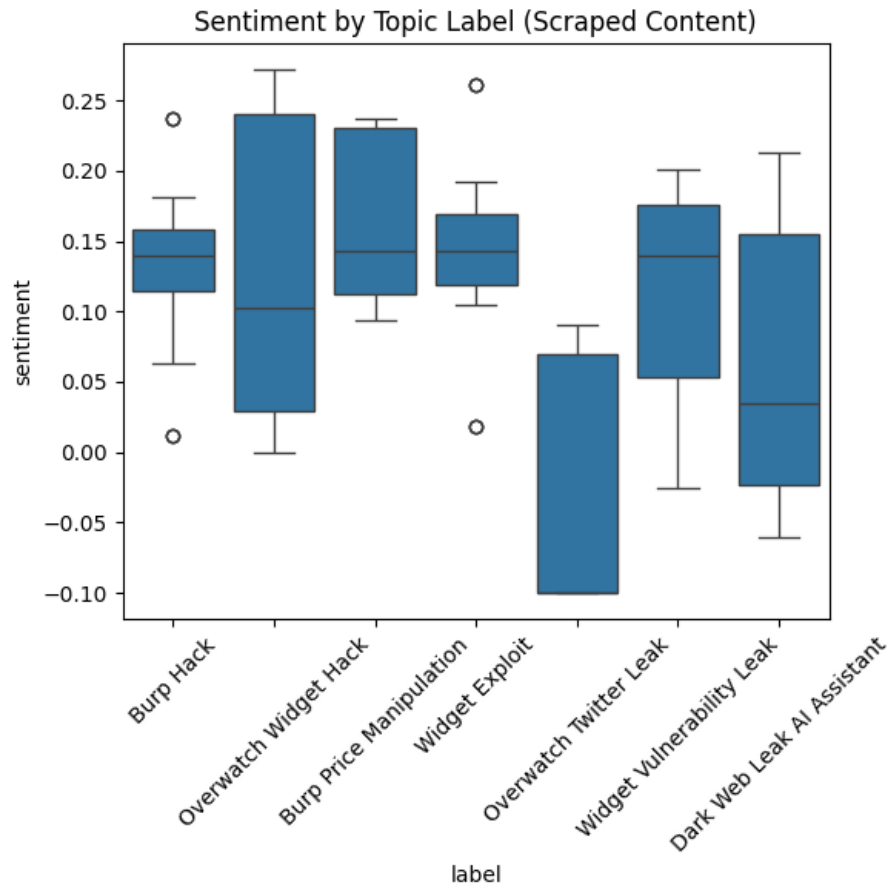


Figure 18: Baseline Sentiment Distribution by Topic Label (Scraped Page Content)

This distribution shows a broader range of sentiment, revealing a mix of positive commentary, neutral technical write-ups, and some negative discussion.

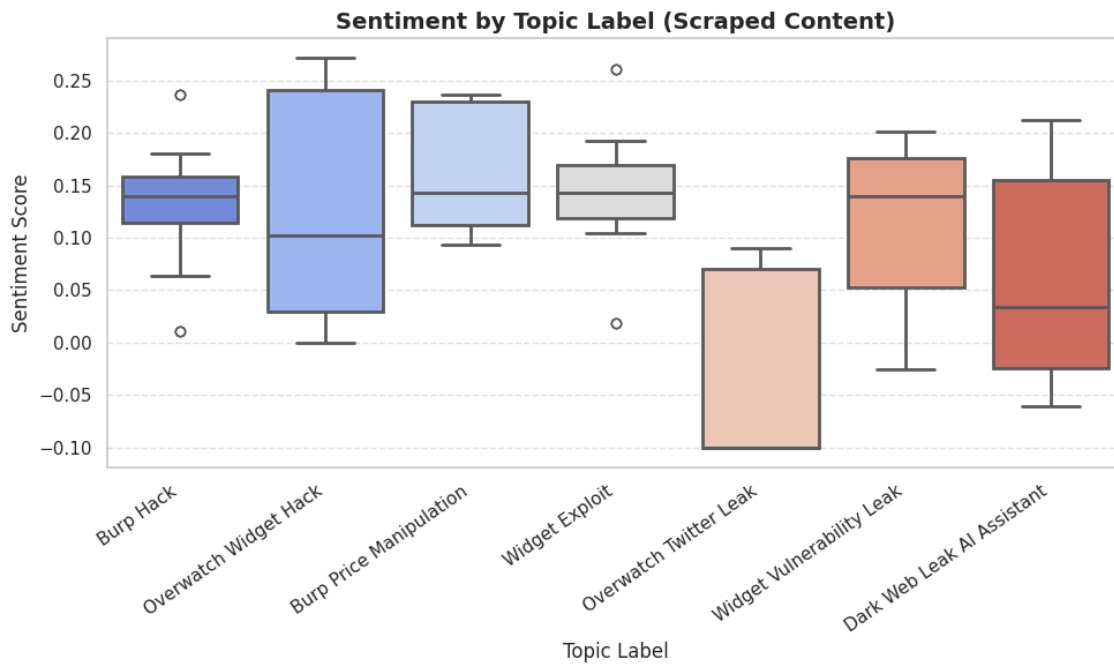


Figure 19: Sentiment Distribution by Topic Label with Mean Sentiment Markers

The box-plot above reveals sentiment trends by topic. For example:

- **Burp Price Manipulation** has the highest average sentiment, suggesting it may be referenced in educational or tool-based walkthroughs.
- **Overwatch Twitter Leak** is the only topic with a negative mean sentiment, likely reflecting criticism or concern.
- **Dark Web Leak AI Assistant** shows a wider spread with some negative outliers, indicating controversy or alarm.

To aid interpretation, I added gold stars representing the mean sentiment for each category and labeled them numerically. I also included a horizontal annotation line to show the overall mean sentiment for reference.

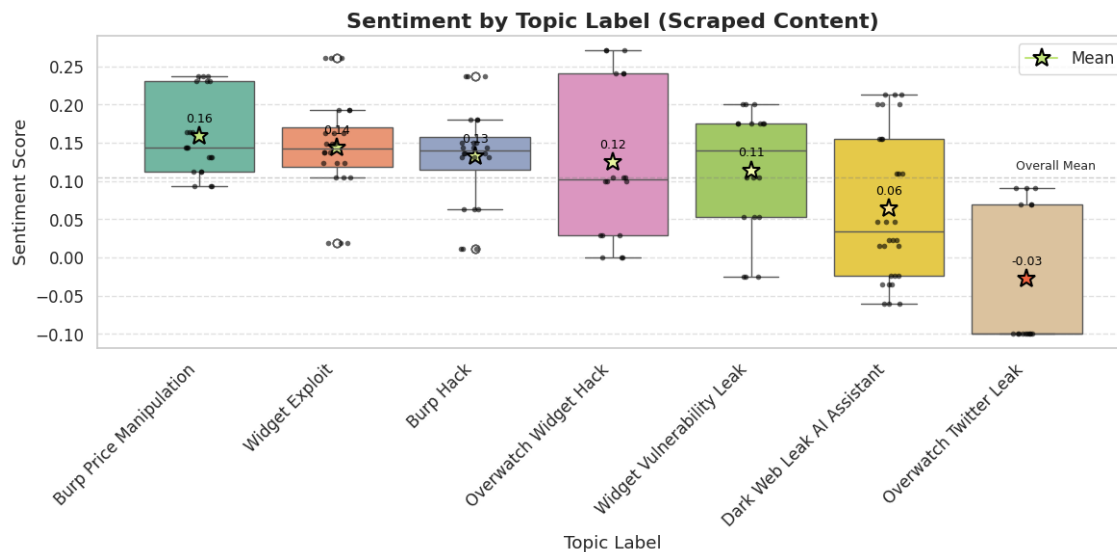


Figure 20: Final Sentiment Distribution by Topic Label with Mean Stars

These results help demonstrate how different aspects of the leak were discussed across online platforms. For instance, while the "Overwatch Widget Hack" query surfaced technical walkthroughs and cheats (with mixed tone), the "Widget Vulnerability Leak" label was generally positive, indicating it may have been viewed as an insightful security finding.

Together, this suggests that while public platforms did not contain explicit mentions of the exact exploit, similar themes and tools (Burp Suite, AI assistants, pricing manipulation) were actively discussed in security and hacker circles. The presence of related terms and sentiment trends may point to indirect exposure or shared methodology.

3.4 Data Science Conclusion

These data science methods allowed for broader environmental scanning of public web discussions. While direct evidence of the original social media post was not found, the data supports the idea that the exploit methodology was publicly accessible and that similar hacks were circulating at the time, potentially increasing the risk of harm to Widget Manufacturing Company's reputation.

Additionally, the sentiment analysis shows that while some labels were viewed neutrally or positively—especially when framed as technical write-ups or educational tools—others (such as the "Overwatch Twitter Leak" and "Dark Web Leak AI Assistant") triggered more negative sentiment, indicating alarm, concern, or criticism. This suggests that the public's reaction to the leak likely varied depending on how and where the content was shared.

Overall, even without finding the original post, this analysis revealed an ecosystem of related discussions and sentiment that could influence public perception and trust. These findings can help guide how the company monitors and responds to future vulnerabilities—especially in fast-moving online spaces.

4 Task 3: Security Audit

In this final task, I evaluated the broader security posture of the Widget Manufacturing Company's purchase system. Building on the earlier exploit test, I used Burp Suite and HTTP inspection tools to identify risks related to client-side price manipulation, sensitive data handling, and weak confirmation logic. My recommendations focus on both immediate fixes and longer-term security improvements.

4.1 Observed Vulnerability: Client-Side Price Manipulation

The main vulnerability lies in trusting sensitive values like product price on the client side. This is in violation of the principle of secure design, which mandates that critical variables, like price, should only be handled on the server. According to OWASP, client-side trust can easily be broken by interception tools like Burp Suite [4].

```
Request
Pretty Raw Hex
1 POST /order/order.php HTTP/2
2 Host: contest.securelyyoursllc.com
3 Content-Length: 162
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Not:A-Brand";v="24", "Chromium";v="134"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: https://contest.securelyyoursllc.com
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/134.0.0.0 Safari/537.36
13 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image
/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: https://contest.securelyyoursllc.com/order/
19 Accept-Encoding: gzip, deflate, br
20 Priority: u=0, i
21
22 product=AI+Assistant&price=299&quantity=1&customer_name=[REDACTED]&customer_email=
[REDACTED]&credit_card=[REDACTED]&order_instructions=
Excited%21
```

Figure 21: Request parameters from Burp Suite showing price set on the client side

4.2 Sensitive Data Exposure in Requests

From the HTTP request inspection in Burp Suite, it is evident that full credit card numbers, customer names, and emails were present in the intercepted request. These values are redacted in the figures for public sharing. This violates PCI DSS recommendations, which advise against handling full card numbers without implementing protections such as tokenization or masking. Tokenization involves replacing sensitive data, such as a credit card number, with a randomly generated, non-sensitive equivalent called a token, which has no exploitable meaning outside the system. Masking, on the other hand, refers to obscuring parts of the data (e.g., displaying only the last four digits of a card number) so that it is unreadable to unauthorized parties. The lack of either measure in this application exposes customers to unnecessary risk.

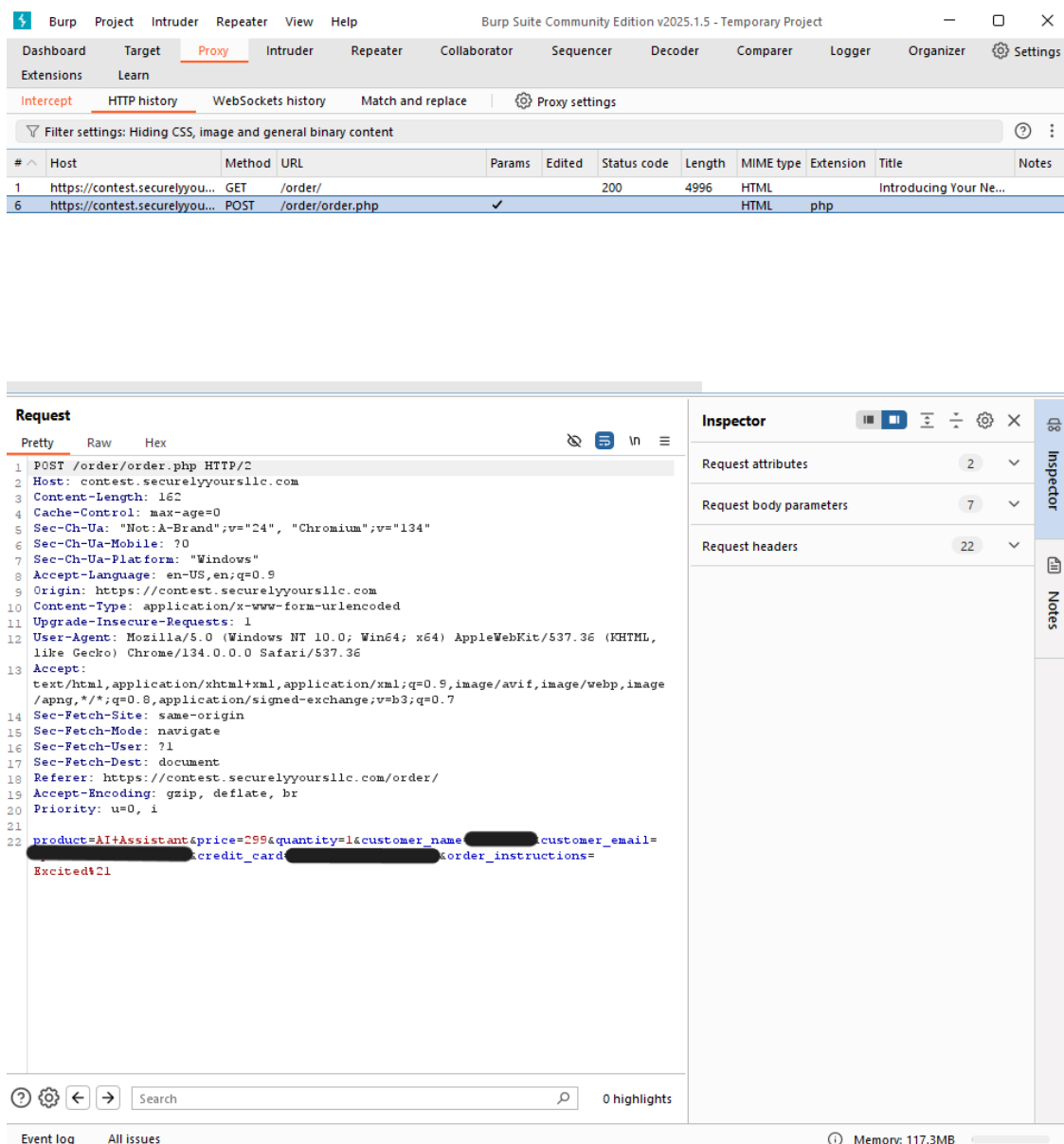


Figure 22: Redacted raw request view showing sensitive personal and payment fields present in the request body

4.3 No Use of Secure Payment Gateway

I did not observe evidence in the request headers or body that a third-party payment processor was being used.

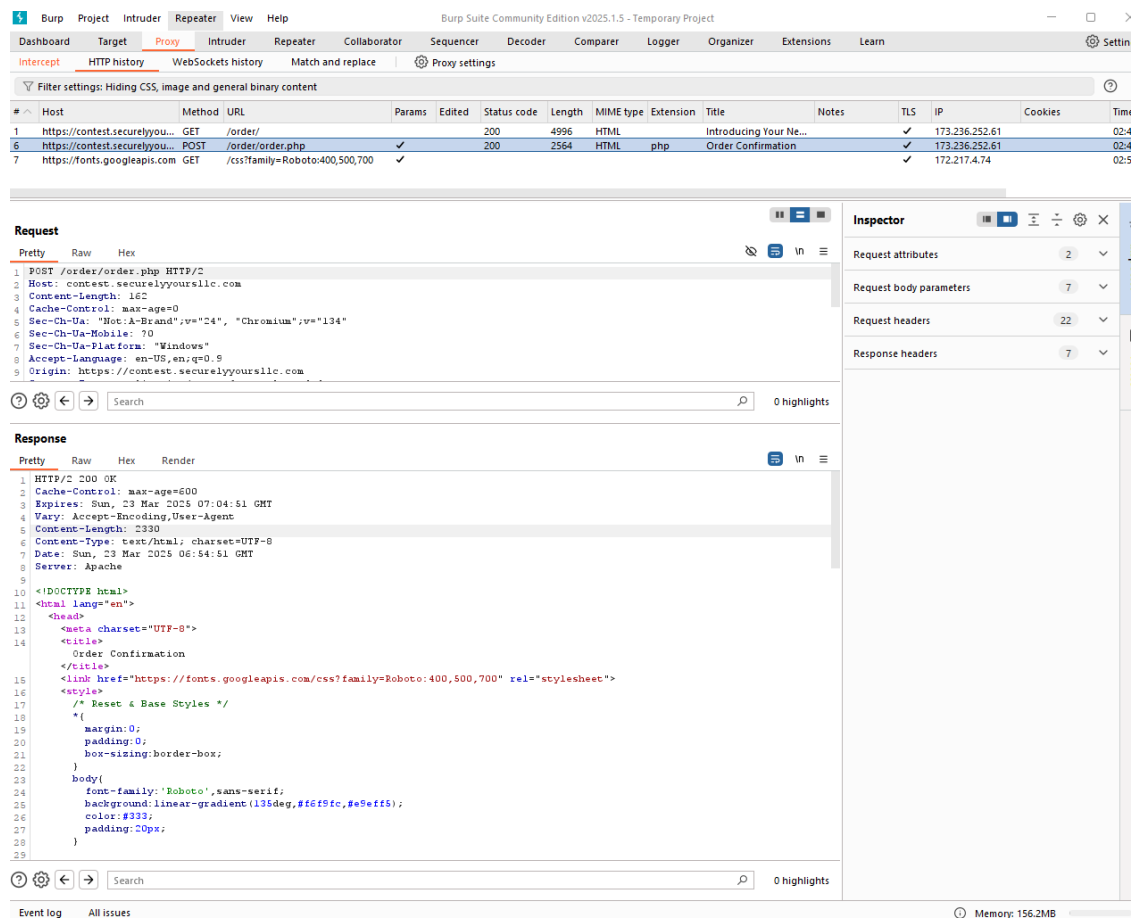


Figure 23: Request headers lack signs of tokenized payment handling

4.4 Unmasked Data in Order Confirmation

After the order was submitted, the confirmation output displayed sensitive customer and payment-related values back to the user. Even when this data is part of a test environment, returning full customer details or full payment values in a confirmation page is unnecessary and increases exposure risk. Order confirmations should only display the minimum information needed, such as an order number, product name, masked email, and the last four digits of a payment card.

This issue reinforces the need to limit reflected user data in server responses and confirmation pages. Sensitive fields should be masked, tokenized, or excluded entirely from user-facing responses.

4.5 Security Recommendations

I have compiled a list of recommendations that are supported by industry best practices:

- **Server-Side Price Calculation:** Never rely on the client to set prices. Instead, use product IDs and let the backend server look up the correct price. This helps prevent tampering and follows OWASP's input validation guidance [5].
- **Use Secure Payment Gateways:** Do not process payments directly. Use trusted third-party services like Stripe or PayPal, which securely handle and tokenize card data [7].
- **Data Masking:** Mask sensitive data such as credit card numbers and email addresses. For example, show only the last 4 digits of a card (e.g., **** * 1234) or obscure emails (e.g., j***@domain.com).
- **HTTPS with TLS 1.3:** Use modern encryption to protect all data sent between users and the server. TLS 1.3 ensures privacy and guards against interception.

- **Add Security Headers:** Apply HTTP headers like "Content-Security-Policy", "Strict-Transport-Security", and "X-Content-Type-Options" to block attacks.
- **Remove Reflected Sensitive Data:** Don't include full user data in responses or confirmations. Only show what's necessary, like order IDs or masked names.
- **Rate Limiting and Logging:** Set limits on how often users can submit requests, and log unusual activity. Tools like `fail2ban` or web application firewalls (WAFs) can help detect and block attacks.
- **Digital Footprint Monitoring:** Use tools like Google Alerts (a free notification system that emails you when your chosen keywords appear online) or CybelAngel (a commercial threat intelligence platform that monitors surface, deep, and dark web sources for data leaks and brand exposure) to watch for leaks, exposed data, or mentions of a company on public forums and dark web sources [1].

These enhancements address both the immediate client-side exploit and broader attack surfaces seen in modern web apps.

References

- [1] CybelAngel. Digital risk protection, 2025. Accessed: 2025-03-23.
- [2] JustAnotherArchivist. snsrape, 2025. Accessed: 2025-03-21.
- [3] Kali Linux. Burp suite - kali linux tools, 2025. Accessed: 2025-03-21.
- [4] OWASP. Broken access control, 2021. Accessed: 2025-03-23.
- [5] OWASP. Secure coding practices - quick reference guide, 2025. Accessed: 2025-03-23.
- [6] PortSwigger. Burp suite - application security testing software, 2025. Accessed: 2025-03-21.
- [7] Stripe. Integration security guide, 2025. Accessed: 2025-03-23.